

Simulating Non-Stationary Gaussian Noise

1 Overview

The script `simulate_nonstationary_noise.py` generates time-domain realizations of Gaussian noise with a prescribed dynamic spectrum $S(f, t)$. The output is a two-column text file containing time in seconds and the simulated noise sample. The default settings generate 256 seconds of data sampled at 2048 Hz, which is a useful size for short LIGO-like examples.

The construction follows the Gram-factor view of the covariance matrix. A positive dynamic spectrum is written as

$$S(f, t) = s^2(f, t),$$

and the amplitude $s(f, t)$ is used as a covariance square root. In a discrete Fourier representation the corresponding covariance has the form

$$C_{ml} = \frac{1}{N} \sum_{k=0}^{N-1} \tilde{s}_k[\{m - k\}_N] \tilde{s}_k^*[\{l - k\}_N],$$

where $\tilde{s}_k$ is the discrete Fourier transform of $s(f_k, t_n)$, and $\{j\}_N$ denotes j modulo N . This can be written as $C = AA^\dagger$, so the covariance is positive semidefinite by construction.

The script provides three simulation strategies: a fast path for separable dynamic spectra, a short-FFT harmonic path for structured nonseparable spectra, and exact direct-sum paths for smaller tests.

2 Default Spectrum

The default frequency-dependent part of the spectrum is

$$S_f(f) = S_0 \left[\left(\frac{f_0}{|f| + f_x} \right)^4 + a \left(\frac{f_0}{|f| + f_x} \right)^{2.5} + b + c \left(\frac{|f|}{f_0} \right)^2 \right],$$

with default values

$$S_0 = 10^{-46}, \quad f_0 = 150 \text{ Hz}, \quad f_x = 10 \text{ Hz},$$

and

$$a = b = c = 1.$$

The absolute value of frequency is used so that the same model can be evaluated on the two-sided FFT frequency grid. The small offset f_x prevents the low-frequency terms from diverging at $f = 0$.

The default time modulation for the separable model is

$$S_t(t) = 1 + A \cos \left(\frac{2\pi t}{\tau} \right),$$

with

$$A = 0.25, \quad \tau = 30 \text{ s.}$$

The code requires $|A| < 1$ so that the spectrum remains positive.

3 Fast Separable Simulation

For separable dynamic spectra,

$$S(f, t) = S_f(f) S_t(t),$$

the covariance square root factors into a stationary frequency coloring and a time-domain amplitude modulation. The simulation is:

1. draw Hermitian Fourier coefficients with variance set by $S_f(f)$;
2. transform the stationary colored noise to the time domain;
3. multiply the time series by $\sqrt{S_t(t)}$.

This costs one inverse FFT and is the default mode:

```
python3 simulate_nonstationary_noise.py
```

The default $S_f(f)$ is interpreted as a one-sided PSD, in units of strain²/Hz. With NumPy's unnormalized FFT convention, the internal DFT-bin variance is

$$S_{\text{internal}}(f) = \frac{f_s}{2} S_1(f),$$

where f_s is the sample rate and $S_1(f)$ is the one-sided PSD. This conversion can be disabled with `--psd-convention internal`.

4 Nonseparable Phase-Band Model

The built-in nonseparable example is a frequency-localized modulation with a frequency-dependent phase:

$$S(f, t) = S_f(f) \left[1 + A G(f) \cos \left(\frac{2\pi t}{\tau} + \psi(f) \right) \right],$$

where

$$G(f) = \exp \left[-\frac{(|f| - f_c)^2}{2\sigma_f^2} \right],$$

and

$$\psi(f) = \frac{2\pi(|f| - f_c)}{\Delta f_{\text{phase}}}.$$

The default values are

$$f_c = 150 \text{ Hz}, \quad \sigma_f = 50 \text{ Hz}, \quad \Delta f_{\text{phase}} = 120 \text{ Hz}.$$

The time average of this model is close to $S_f(f)$, while different frequency bands reach their maxima at different times. This creates a simple nonseparable dynamic spectrum without changing the mean spectrum very much.

5 Short-FFT Harmonic Path

The phase-band model can be simulated quickly because its square-root modulation has low harmonic rank in time. The method assumes that

$$\sqrt{\frac{S(f, t)}{S_f(f)}} \simeq \sum_{h=-H}^H a_h(f) e^{ih\Omega t}$$

for a modest value of H . Each retained harmonic can then be simulated using FFT operations, so the cost is roughly proportional to the number of retained harmonics rather than N^2 .

Slow, smooth modulation is one way to make this approximation efficient, but the essential requirement is low harmonic content in time. A localized frequency band $G(f)$ is useful for modeling localized non-stationarity, but it is not the primary speed requirement. Broad-band modulation is still fast if only a few harmonics are needed. Sharp transient time dependence generally requires many harmonics or a different structured basis.

The harmonic engine is selected automatically for the phase-band model when `--mode general` is used:

```
python3 simulate_nonstationary_noise.py --mode general
```

It can also be requested explicitly:

```
python3 simulate_nonstationary_noise.py --mode general --general-engine harmonic
```

Convergence should be checked by increasing `--harmonic-count` and `--harmonic-quadrature`. The defaults are

$$H = 8, \quad N_{\text{quad}} = 256.$$

6 Exact Direct-Sum Paths

For a completely general dynamic spectrum, the exact simulation evaluates the covariance-square-root construction directly. Schematically,

$$x_n = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} s(f_k, t_n) \eta_k e^{2\pi i k n / N},$$

where the η_k are independent unit-variance Fourier-basis random variables, with Hermitian symmetry imposed to produce a real time series.

This direct method scales as N^2 , so it is intended mainly for small tests. The pure NumPy implementation calls the user-editable function `example_dynamic_psd()`:

```
python3 simulate_nonstationary_noise.py --mode general --general-engine numpy \
--duration 1 --sample-rate 512
```

If numba is installed, the direct sum can be JIT accelerated:

```
python3 simulate_nonstationary_noise.py --mode general --general-engine numba
```

However, numba cannot compile arbitrary Python callbacks. A user who wants a new dynamic spectrum to benefit from numba acceleration should write the model in jit-able scalar form, either by modifying the existing `_simulate_general.builtin_numba()` kernel or by adding a new `@njit` kernel.

7 Command-Line Options

The most commonly used options are:

- `--duration` Duration in seconds. Default: 256.
- `--sample-rate` Sample rate in Hz. Default: 2048.
- `--seed` Random seed. Default: 12345.
- `--output` Output file. Default: `nonstationary_noise.dat`.
- `--mode` Either `separable` or `general`. Default: `separable`.
- `--dynamic-example` Dynamic-spectrum example for general mode: `separable` or `phase-band`. Default: `phase-band`.
- `--general-engine` Engine for general mode: `auto`, `harmonic`, `numpy`, `numba`, or `separable-fast`. Default: `auto`.
- `--psd-convention` Use `one-sided` for a one-sided PSD in $\text{strain}^2/\text{Hz}$, or `internal` for the DFT-bin convention.
- `--fft-backend` Use `auto`, `numpy`, or `scipy`. The `auto` setting prefers SciPy when available.

The default spectrum parameters can be changed with `--s0`, `--f0`, `--fx`, `--a`, `--b`, and `--c`. The time modulation parameters are `--mod-amp` and `--mod-period`. The phase-band parameters are `--phase-band-fc`, `--phase-band-sigma-f`, and `--phase-band-delta-f-phase`.

8 Examples

Generate the default separable LIGO-like example:

```
python3 simulate_nonstationary_noise.py
```

Write a different output file and random realization:

```
python3 simulate_nonstationary_noise.py --seed 7 --output ligo_like_noise.dat
```

Generate the nonseparable phase-band example using the harmonic engine:

```
python3 simulate_nonstationary_noise.py --mode general \  
--dynamic-example phase-band --general-engine harmonic \  
--output phase_band_noise.dat
```

Run a small exact check of the nonseparable model:

```
python3 simulate_nonstationary_noise.py --mode general \  
--dynamic-example phase-band --general-engine numpy \  
--duration 1 --sample-rate 512
```

9 Editing the Model

For most new separable examples, edit the functions

```
example_frequency_psd()    example_time_modulation() .
```

For a nonseparable model, edit

```
example_dynamic_psd() .
```

To produce a real time series, the dynamic spectrum should be positive and even in frequency,

$$S(-f, t) = S(f, t).$$

If the new model has low harmonic rank in time, it may be possible to add a specialized short-FFT engine similar to the phase-band implementation. If the model is fully general and the exact direct sum is needed at larger N , the model should be implemented in a numba-compatible kernel.

As an example, generate 256s of non-stationary noise using the phase band option:

```
python3 simulate_nonstationary_noise.py --mode general\  
--dynamic-example phase-band --duration 256\  
--sample-rate 2048 --output phase_band_noise.dat
```

Run the output through the BayesWave spectral estimation code to produce the periodogram and PSD estimate shown below: Next run the output from BayesWave through the WDM wavelet

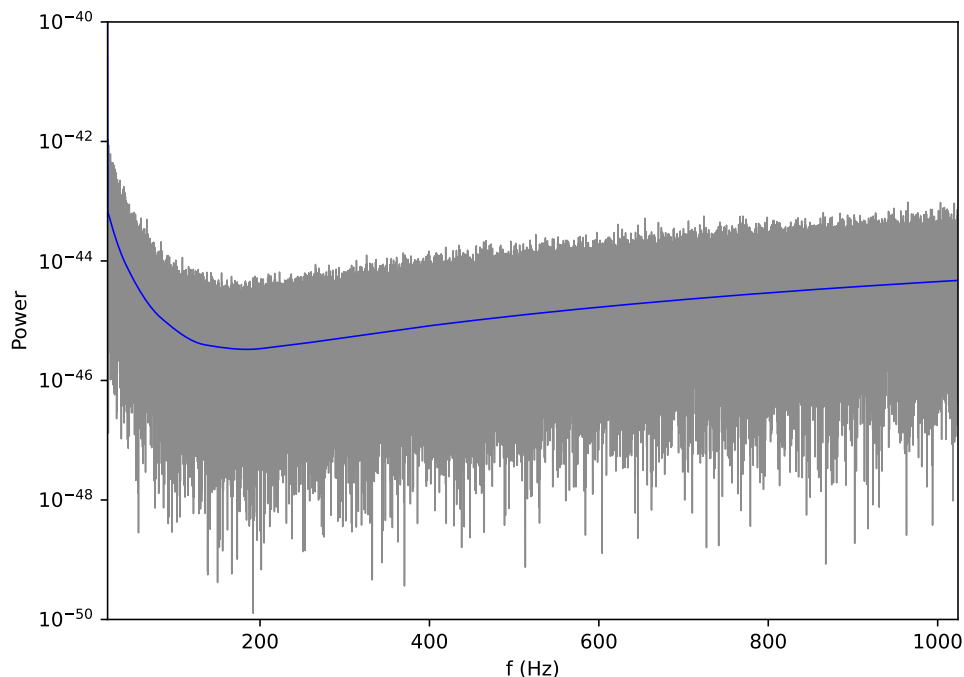


Figure 1: Periodogram and Bayeswave spectral estimate for phase_band_noise.dat

analysis suite to produce a smoothed power map with the mean subtracted. This shows the non-stationary behavior in the 150 Hz band. The upper panel in the plot shown below is the average

power (averaged over frequency) at each time. This diagnostic alone does not pick up the non-stationary behavior since it is confined to a small band. But visually we can see the non-stationarity in the variance map.

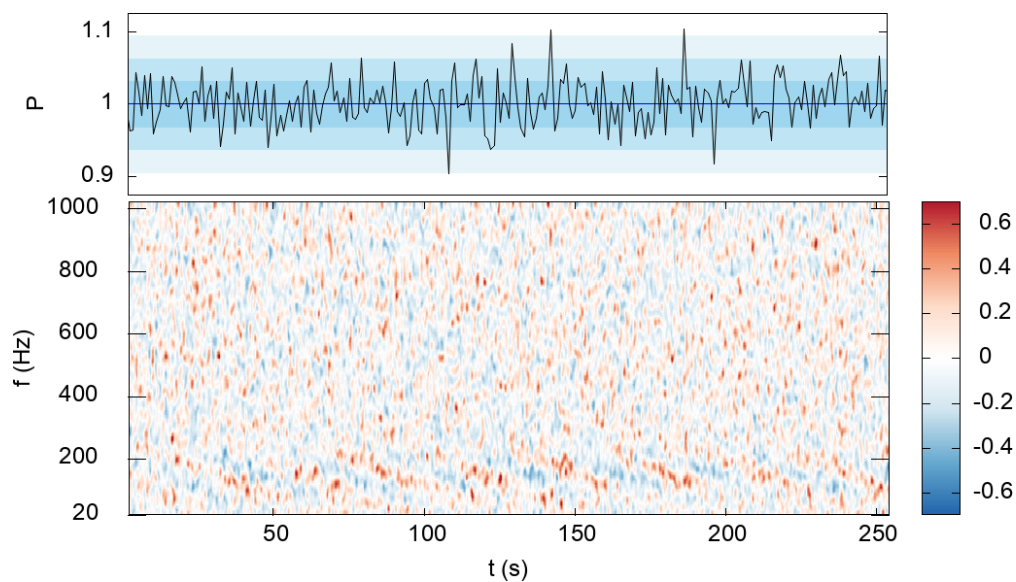


Figure 2: The upper panel shows the average power as a function of time. This falls within the range one would expect for stationary noise. But the mean-variance-subtracted smoothed power map shows clear signs of non-stationarity in the 150 Hz band. A Qscan would also pick this up.